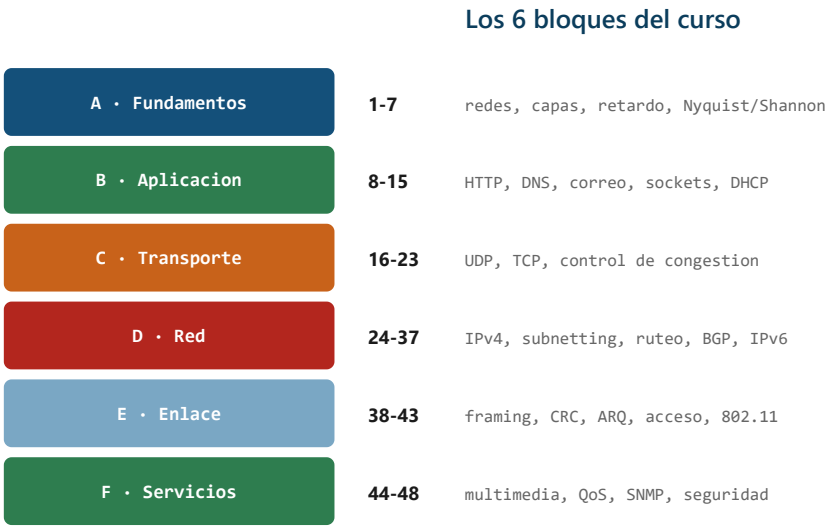
Clase 23 — Repaso de transporte + Simulacro 3



Bloque: C (Transporte) · **Semana:** 8 · **Duración estimada:** 120 min (repaso 50 + simulacro 70) **Correlativa:** Clase 22 (taller TCP) · **Sigue con:** Clase 24 (la capa de red: las dos funciones) **Material de referencia:** Tanenbaum cap. 6 completo · RFC 9293, RFC 5681, RFC 9438, RFC 2018, RFC 7323, RFC 6928, RFC 3168

🎯 Al terminar esta clase vas a poder

1. Reconstruir de memoria el mapa de las **8 clases** del bloque de transporte (16 a 23).
2. Escribir **de memoria** las fórmulas que aparecen en los exámenes de transporte.
3. Decir los **11 estados** de la máquina de TCP y la condición de cada transición.
4. Completar una tabla **TCP vs. UDP** de 12 filas sin mirar.
5. Identificar y corregir **las 7 confusiones** que más se castigan en este bloque.
6. Resolver un examen completo de 100 puntos en 3 horas: 5 teóricas + 2 problemas.

> **Prerequisito del examen: TODOS.** Esta clase cierra el bloque C. El simulacro reproduce el formato real (`2011-03` , `2010-12` , `2011-02`): 5 teóricas + 2 problemas de 30 pts, 3 horas, **mínimo 15 puntos en teoría para aprobar**.

PARTE 1 — REPASO SINTÉTICO DEL BLOQUE

1.1 El mapa del bloque

```
┌──────────────────────────────────────────────────────────────────┐
│ 16 MULTIPLEXACIÓN        2-tupla UDP vs 4-tupla TCP · mux/demux  │
│ 17 CABECERA TCP          campos · flags · opciones · MSS         │
│ 18 ESTABLECER/CERRAR     handshake 3-vías · 11 estados · TIME-WAIT│
│ 19 FIABILIDAD            seq · timers · RTO · BDP · SACK · PSH   │
│ 20 FLUJO                 rwnd · min(cwnd,rwnd) · zero window     │
│ 21 CONGESTIÓN            slow start · CA · fast recovery · ECN   │
│ 22 TALLER                diagramas de intercambio resueltos      │
│ 23 REPASO + SIMULACRO                                            │
└──────────────────────────────────────────────────────────────────┘

UDP = una carta      TCP = telegrama certificado con acuse de recibo

Las garantías de TCP se COMPRAN con tres mecanismos:
  números de secuencia → orden, sin duplicados, saber qué reenviar
  timers + RTO         → saber CUÁNDO reenviar
  ventana deslizante   → no esperar el ACK antes de seguir mandando
y se PAGAN con dos controles:
  rwnd → no desbordar al receptor
  cwnd → no saturar la red
```

1.2 Las fórmulas que hay que saber de memoria

```
① RTO = SRTT + 4 · RTTVAR
       SRTT   ← (1−α)·SRTT + α·R              con α = 7/8
       RTTVAR ← (1−β)·RTTVAR + β·|SRTT − R|    con β = 1/4
       1ª muestra: SRTT = R, RTTVAR = R/2     (piso y techo)

② BDP = ancho_de_banda × RTT          [bytes] = [bits/s] × [s] / 8

③ ventana efectiva = min(cwnd, rwnd)         ← MÍNIMO, nunca suma

④ throughput = min( ventana_efectiva / RTT , ancho_de_banda_del_enlace )

⑤ MSS = MTU − 20 (IP) − 20 (TCP) = 1500 − 40 = 1460   (1448 con timestamps)

⑥ tiempo = tamaño_del_archivo / throughput

⑦ ataque TCP reset:  paquetes = 2³² / ventana = 2³² / 2¹⁴ = 2¹⁸ = 262 144
                     si no se conoce el puerto:  × 2¹⁶ = 65 536 puertos

⑧ módulos: puerto 16 bits → 2¹⁶ = 65 536 (0…65 535)
           número de secuencia 32 bits → 2³² = 4 294 967 296

⑨ d_trans = L / R
```

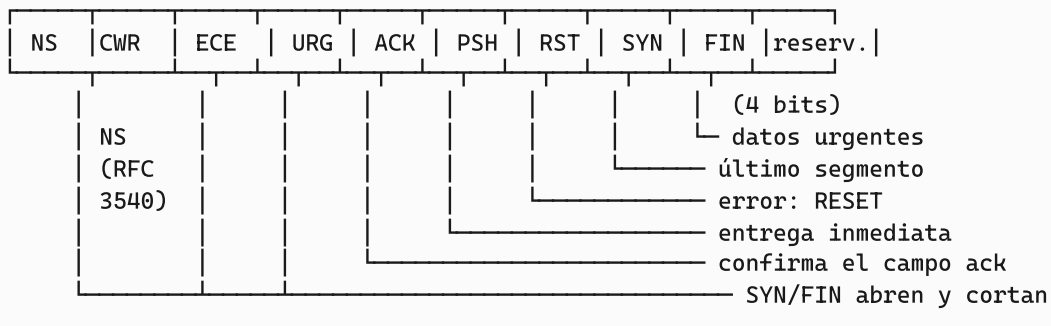
1.3 Los puertos que hay que saber

Puerto	Servicio	Proto	Por qué lo importan
20 / 21	FTP datos / control	TCP	dos puertos, control separado
22	SSH	TCP	también `22/udp` y `22/sctp` en IANA
25	SMTP	TCP	envío, *push*
53	**DNS**	**UDP y TCP**	⚠️ los dos: consultas y respuestas grandes / *zone transfer*
67 / 68	DHCP server / client	UDP	⚠️ **nunca TCP**: el cliente aún no tiene IP
80	HTTP	TCP	también `80/udp` y `80/sctp`
110 / 995	POP3 / POP3S	TCP	*pull*, borra del servidor
143 / 993	IMAP / IMAPS	TCP	*pull*, sincroniza
161 / 162	SNMP / trap	**UDP**	dos puertos: consulta y notificación
179	BGP	TCP	⚠️ un protocolo de **enrutamiento** sobre transporte **fiable**
443	**HTTPS**	**TCP y UDP**	⚠️ el UDP es **HTTP/3 (QUIC)**
853	DNS sobre TLS (DoT)	TCP	DNS cifrado
546 / 547	DHCPv6 client / server	UDP	el DHCP de IPv6

```
⚠️ 53 UDP+TCP   → si solo decís "DNS usa UDP", te falta la mitad
⚠️ 443 TCP+UDP  → el UDP es HTTP/3; HTTP/3 NO usa TCP para los datos
⚠️ 179 TCP      → BGP necesita fiabilidad
⚠️ 67/68 UDP    → DHCP en UDP porque el cliente aún no tiene dirección

Rango: 16 bits ⇒ 0 … 65 535.   0…1023 = bien conocidos / privilegiados
       (en Unix, `bind()` sobre ellos requiere root)
```

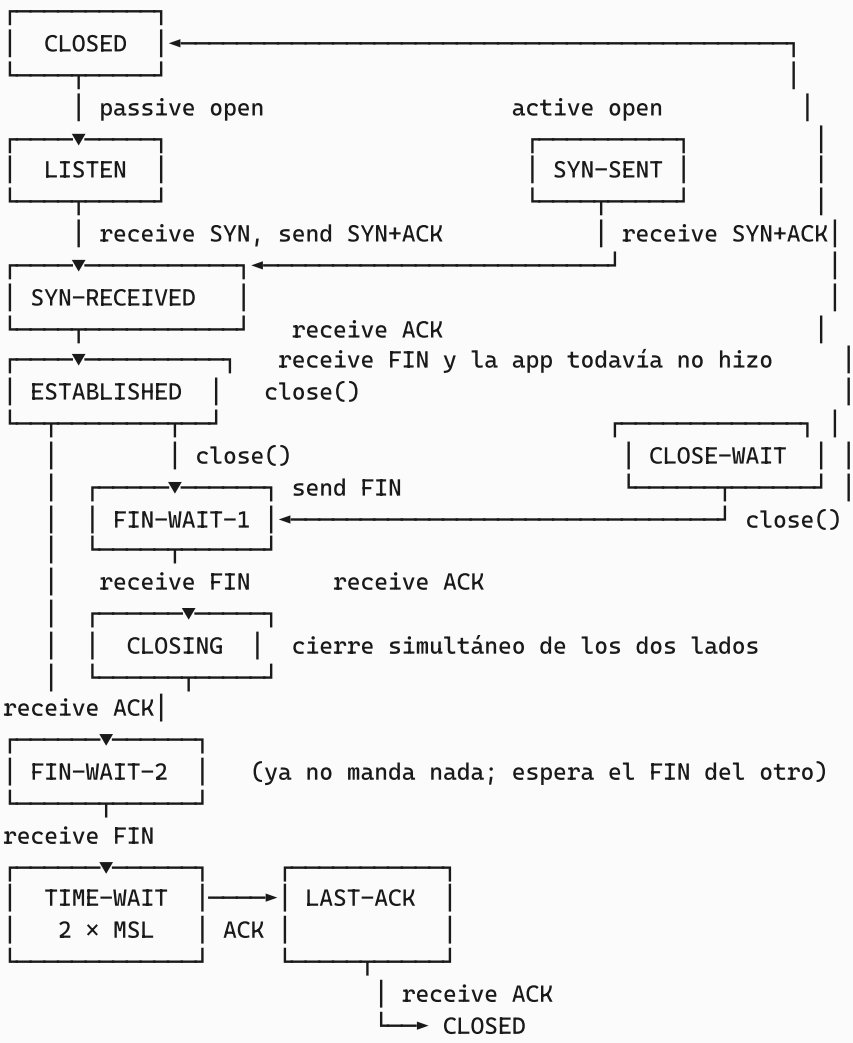
1.4 Los flags TCP



Flag	Significado	Dónde se usa
SYN	sincronizar	abre la conexión (handshake)
ACK	confirmar	`1` en casi todo lo que sigue al handshake
FIN	finalizar	media conexión de cierre
RST	reset	**corta la conexión de inmediato**, sin negociación
PSH	push	sugiere entregar ya a la aplicación (**no garantiza nada**)
URG	urgente	activa el *urgent pointer*
ECE / CWR	control de congestión	**ECN** (`RFC 3168`): ECE=1 "hubo congestión", CWR=1 "bajé mi ventana"
NS	no-sacudir	extensión: no limpiar el campo ECN al calcular el checksum

```
★ El flag PSH NO garantiza un mensaje por read() (Clase 19 §1.8, 2001-12 Prob. 2)
★ RST no se negocia: aborta sin cierre ordenado
★ SYN y FIN abren/cortan; ACK y PSH son de "tránsito"
★ ECE/CWR existen SOLO por ECN; sin ECN van en 0
```

1.5 Los 11 estados de la máquina de TCP



Los once, numerados: CLOSED, LISTEN, SYN-SENT, SYN-RECEIVED, ESTABLISHED, FIN-WAIT-1, FIN-WAIT-2, CLOSE-WAIT, CLOSING, LAST-ACK, TIME-WAIT.

```
★ TIME-WAIT = 2 × MSL  (Maximum Segment Lifetime)
  ¿Por qué 2×MSL y no 1? Un segmento perdido puede reaparecer como duplicado
  tarde. Si el socket se cerrara a los MSL, ese duplicado con la misma
  4-tupla caería dentro de una conexión NUEVA y corrompería el flujo.
  Con 2×MSL se garantiza que el viejo ya expiró en la red.

★ LAST-ACK: mandó su FIN y espera el ACK del FIN ajeno. No es lo mismo que
  TIME-WAIT: acá ya no manda nada.
★ CLOSING: cierre simultáneo. Raro, y es el que hay que saber dibujar.
★ Un RST en cualquier estado → CLOSED, sin negociación y sin TIME-WAIT.
★ Costo: TIME-WAIT retiene un puerto por 2×MSL ⇒ agotamiento de puertos
  efímeros en servidores con muchos clientes cortos.
```

1.6 La tabla TCP vs. UDP — 12 filas

#	Aspecto	UDP	TCP
1	**Conexión**	sin conexión (*connectionless*)	**con conexión**, 3-vías
2	**Fiabilidad**	**no**: se pierde y sigue	**sí**: ACK + timers + retransmisión
3	**Orden**	**no garantizado**	**sí**, por número de secuencia
4	**Latencia**	**baja**, sin handshake	**1 RTT extra** de handshake
5	**Control de congestión**	**ninguno** (¡por eso QUIC!)	**sí**: `cwnd`, slow start, ECN
6	**Control de flujo**	ninguno	**sí**: `rwnd`
7	**Cabecera**	**8 bytes** (2+2+2+2)	**20 bytes** mínimo, hasta 60
8	**Identificación**	**2-tupla** (IP, puerto) *(el kernel usa la 5-tupla)*	**4-tupla** (IP_o, p_o, IP_d, p_d)
9	**Multiplexación**	**1 socket habla con muchos** a la vez, mismo puerto	**1 conexión por 4-tupla**: 1 socket por conexión
10	**Checksum**	obligatorio en IPv6, opcional en IPv4	**siempre obligatorio**
11	**Frontera de mensaje**	**SÍ**: 1 datagrama = 1 mensaje	**NO**: flujo de bytes; los mensajes los pone la aplicación
12	**Usos típicos**	DNS, DHCP, SNMP, RTP/VoIP, QUIC (HTTP/3)	HTTP/1.1 y HTTP/2, SSH, SMTP, POP3/IMAP, FTP, BGP

Las tres filas donde más se falla:

```
Fila 8 — 2-tupla vs 4-tupla:
  UDP  → el socket se identifica con (IP, puerto) ⇒ un solo proceso
         (el servidor DNS, puerto 53) atiende a TODOS los clientes.
  TCP  → cada 4-tupla distinta necesita un socket distinto ⇒ el SO crea
         uno por accept().

Fila 11 — frontera de mensaje:
  UDP  → cada datagrama es una unidad: leés N bytes, es UN mensaje.
  TCP  → read() puede devolver medio mensaje o dos pegados. PSH no lo
         arregla (Clase 19 §1.8).

Fila 5 — por qué HTTP/3 necesita QUIC:
  UDP solo no tiene control de congestión, y eso sería grave en Internet.
  QUIC (RFC 9000/9002) reimplementa fiabilidad, control de congestión y
  PLPMTUD ENCIMA de UDP. No es "HTTP/3 usando UDP": es un transporte
  nuevo que compite con TCP.
```

1.7 Las tres tablas de memoria en una sola vista

```
¿Qué puerto?              ¿Qué flags?              ¿En qué estado?
──────────────            ──────────────           ──────────────────────
DNS     53 UDP/TCP        handshake: SYN,          CLOSED → LISTEN (servidor)
HTTP    80 TCP            SYN+ACK, ACK              SYN-SENT → SYN-RECEIVED
HTTPS  443 TCP/UDP        datos: ACK, PSH,          → ESTABLISHED
SSH     22 TCP            cierre: FIN, ACK          ESTABLISHED → FIN-WAIT-1
SMTP    25 TCP                                      → FIN-WAIT-2 → TIME-WAIT
DHCP  67/68 UDP           error: RST                → LAST-ACK → CLOSED
BGP    179 TCP            sync: ECE, CWR           (también: CLOSING)
SNMP 161/162 UDP
```

⚠️ Actualización a 2026

Sigue igual — de memoria, no se toca: - **4-tupla** identifica una conexión TCP; el socket UDP se trata como **2-tupla**. - Los **11 estados** son los mismos. **Three-way handshake**, cierre de **4 vías**, **TIME-WAIT de 2×MSL**. - **Números de secuencia de 32 bits**, **ACK acumulativo**, **ventana deslizante**. - **Los ACKs son de bytes, no de segmentos**: `ack` = próximo byte esperado. - Sliding window, dupACKs, delayed ACK, Nagle, persist/keepalive, RTO, Jacobson/Karels, fast retransmit, slow start, congestion avoidance, fast recovery. - **MTU/MSS**, opciones en el SYN (MSS, SACK permitted, Timestamps, Window Scale), checksum con pseudo-cabecera, descarte de segmentos fuera de ventana.

Lo que cambió — las correcciones obligatorias del bloque:

En 2011 se decía	En 2026
`RFC 793` es TCP	**`RFC 9293`** (2022, *Internet Standard*; consolida 793 + RFC 879, 2873, 6093, 6429, 6528, 6691)
Reno / Tahoe	**CUBIC = `RFC 9438`** (2023, *Standards Track*), default de Linux, Windows y Apple; obsolesce `RFC 8312`
"quizá BBR"	**BBR nunca fue RFC**: `draft-ietf-ccwg-bbr`, BBRv3, *Experimental*. ⚠️ **`RFC 8312` es CUBIC, no BBR**
IW = 1 MSS	**IW = 10 MSS** (`RFC 6928`, 2013)
SACK opcional	`RFC 2018` es **comportamiento estándar**; Timestamps + Window Scale = `RFC 7323` (obsolesce `1323`); **PAWS obligatorio** si se activan
ECN = `RFC 3168`	`RFC 3168` + **`RFC 9768` AccECN (2026)**; L4S = `RFC 9331`
—	**QUIC** (`RFC 9000/9001/9002`, 2021): transporte completo que **reemplaza** a TCP en HTTP/3
—	**MPTCPv1** (`RFC 8684`, 2020): multipath, real pero marginal (~0,4 % de bytes pico)

⚠️ Tres errores que NO son obsolescencia sino directamente falsos:

Se dice	Realidad
"`RFC 8951` = ECN en multihop"	**FALSO**: es una aclaración de **EST** (Enrollment over Secure Transport)
"`RFC 7560` = ECN++"	**FALSO**: es solo el **documento de problema** de ECN; la especificación quedó en `RFC 9768`
"`RFC 8984` depreca el half-duplex Ethernet"	**FALSO**: `RFC 8984` es **JSCalendar**. La decisión es de **IEEE 802.3**, y ni siquiera es total

> **Consecuencia para el examen:** los **mecanismos** se preguntan **exactamente igual** que en 2011. Lo único que se corrige es **la etiqueta normativa** y, si preguntan por despliegue real, quién está en el default (CUBIC, no Reno).

PARTE 2 — LOS ERRORES FRECUENTES

Los siete que más puntos se pierden en los exámenes de transporte.

Error 1 — "El ACK confirma un segmento"

❌ "A mandó S2, entonces A2 confirma S2." ✅ El ACK es **acumulativo y numera bytes**: `ack` = **próximo byte esperado**. Un `ack = 7120` confirma los 120 bytes del 7000 al 7119 de una sola vez. **Por qué importa:** si razonás por segmentos, fallás todos los diagramas con pérdida, que es justo lo que se pregunta (`2011-03 P4`).

Error 2 — "Si se perdió el primero, mando el segundo otra vez"

❌ Ante `ack = 7000` repetido, "el que falta es el primero, así que mando el segundo". ✅ `ack = 7000` significa que **falta a partir del 7000**, o sea el **primero** (S1, `seq = 7000`). Es el que hay que retransmitir. **Por qué importa:** arruina el apartado (b) de `2011-03 P4`.

Error 3 — "El timeout es para detectar la pérdida"

❌ "TCP usa un timer para saber cuándo se perdió un segmento." ✅ El timeout existe porque, si el ACK **no llega**, TCP **no puede distinguir** tres cosas: se perdió el dato, se perdió el ACK, o tardó. Las tres se ven **exactamente igual**. La única salida es retransmitir, y para eso hay que saber **cuándo** → el timer. **Por qué importa:** es el apartado (b) de `2010-12 P4`, la pregunta conceptual más fina.

Error 4 — "Flujo y congestión son lo mismo"

❌ "Ambos bajan la ventana, así que son lo mismo." ✅ **Flujo** = el receptor no puede absorber → `rwnd`, lo controla el **receptor** y lo anuncia en el campo `window`. **Congestión** = la red no puede transportar → `cwnd`, lo controla el **emisor** estimando, y nadie se lo dice. **Por qué importa:** `2009-07 P1a` (10 pts) es esta pregunta. Y si bajás `cwnd` cuando el `rwnd` es 0, **castigás a la red por un problema del receptor** y no arreglás nada.

Error 5 — "La ventana efectiva es cwnd + rwnd"

❌ "Se suman porque son cosas distintas." ✅ Es el **mínimo**: `min(cwnd, rwnd)`. Ambas son **cotas superiores**. Con la suma, el excedente que la red no tomó se acumularía en el buffer del receptor y lo desbordaría **justo cuando la red va peor**. **Por qué importa:** es la mitad de `2011-03 Problema 1c` (12 pts) y aparece en toda `2009-07 P1a`.

Error 6 — "Con PSH el receptor lee un mensaje"

❌ "Puse PSH, así que el `read()` del otro devuelve exactamente el mensaje." ✅ PSH es **una sugerencia** de "no tengo más datos en el buffer". Es **un bit**: no transporta información de frontera de mensaje. Si llegan dos segmentos con PSH **fuera de orden**, el `read()` devuelve **los dos pegados**. Las fronteras las pone la aplicación. **Por qué importa:** es `2001-12 Problema 2` entero, y es la fuente del error de cualquiera que programme sockets.

Error 7 — "La pérdida siempre significa congestión"

❌ "Se perdió un paquete ⇒ la red está saturada." ✅ **No.** También por CRC de un enlace, buffer del receptor lleno (control de **flujo**), ACL/firewall, fragmento descartado en IPv4, corte de un enlace. **TCP asume** que sí, porque es la apuesta con mejor promedio: castigar a la red de vez en cuando sale más barato que saturarla. **Por qué importa:** es `2009-07 P1c`, y revela si entendiste que TCP **infiere** la congestión en lugar de que se la **notifiquen**.

⚠️ Errores típicos del simulacro (los que no son de este bloque)

```
⚠️ No cites "la RFC de TCP" como RFC 793 → es RFC 9293
⚠️ No digas "el timeout baja cwnd a cwnd/2" → a 1 MSS (Tahoe y Reno coinciden)
⚠️ No digas "3 dupACKs ⇒ cwnd = 1 MSS" en Reno ⇒ cwnd/2 (eso es Tahoe)
⚠️ No digas "DNS usa el puerto 53 UDP" solo ⇒ también TCP
⚠️ No digas "la cabecera TCP son 20 bytes" sin aclarar "mínimo"
```

PARTE 3 — SIMULACRO 3

> **Instrucciones (reproducción del formato real de FING):** - Examen individual, **sin material**, **3 horas**. - **5 Preguntas Teóricas** (~40 pts) + **2 Problemas Prácticos** (30 pts cada uno). - Aprobación: **60 puntos**. - ⚠️ **Es obligatorio responder correctamente al menos 15 puntos en las preguntas teóricas.** Si no llegás a 15 en teoría, no aprobás aunque sumes 60 con los problemas. - Empezá cada pregunta y cada problema en una hoja nueva. - Dudas de letra solamente; **ninguna** en los últimos 30 minutos. - Para los problemas podés suponer que disponés de los tipos básicos (lista, cola, archivo, string) y sus funciones (`tail()` , `crear()` , `concatenar()`).
>
> ⏱️ **Cronómetro sugerido (180 minutos):**

Etapa	Min	Objetivo
Lectura inicial	5	leer entero, elegir por dónde empezar
T1 (10 pts)	15	
T2 (10 pts)	15	
T3 (8 pts)	12	
T4 (10 pts)	15	
T5 (8 pts)	12	**69 min de teoría, ~46 pts**
Problema 1 (30 pts)	50	pseudocódigo estilo C: leer bien el enunciado
Problema 2 (30 pts)	45	números: cuentas a la vista, TABLAS antes que prosa
Repaso final	5	re-leer los números de los dos problemas
Total	**180**	

> **El consejo de la cátedra, y es el que más cuesta aprender:** los problemas prácticos se corrigen **por lo que escribiste**, no por lo que pensaste. Si no te da el número exacto, **dejá la fórmula y la cuenta a medias** con la unidad anotada: parcialmente es puntos. En cambio un apartado en blanco es cero.

📝 ENUNCIADO DEL SIMULACRO (sin respuestas)

Preguntas Teóricas

T1 (10 puntos). Un protocolo de transporte entrega a la aplicación un flujo de bytes y promete que **no se pierde nada, no se duplica nada y todo llega en orden**. a) Explique qué mecanismo permite cumplir cada una de esas tres promesas, y por qué el `checksum` por sí solo no alcanza. b) Suponga que el emisor manda un segmento, no recibe el ACK y se vence el timer. Enumere **todas** las explicaciones posibles de ese fallo y explique por qué el protocolo **no puede distinguir entre ellas**. ¿Qué decisión queda, y qué falta para poder tomarla? c) Explique cómo estima el protocolo el valor del timer, y qué ocurre con ese valor tras varios timeouts consecutivos.

T2 (10 puntos). En un protocolo de transporte con control de congestión y de flujo: a) Diferencie el control de flujo del control de congestión, indicando **qué variable** representa a cada uno, **quién** la decide y **cómo** se comunica al otro extremo. b) Escriba la expresión de la ventana efectiva y justifique por qué es un **mínimo** y no una suma. c) El receptor anuncia una ventana de recepción igual a cero. Detalle qué hace el emisor, con qué periodicidad, y justifique por qué esa situación **no** debe tratarse como congestión. d) Mencione dos causas de pérdida de paquetes que **no** se deban a congestión.

T3 (8 puntos). Sobre el protocolo TCP: a) Escriba los **11 estados** de su máquina de estados. b) Detalle la secuencia de **mensajes** del establecimiento de una conexión, indicando qué flags activa cada uno y cómo se numeran los bytes iniciales en ambos sentidos. c) Explique por qué, tras el cierre ordenado, un extremo permanece en `TIME-WAIT` durante `2 × MSL`, y qué ocurriría si no lo hiciera.

T4 (10 puntos). Compare los dos protocolos de transporte de la familia TCP/IP. a) Para cada uno de los siguientes aspectos, indique la diferencia: (i) existencia de conexión; (ii) número de bytes de cabecera; (iii) forma de identificar una comunicación; (iv) manejo de la pérdida de datos; (v) frontera de mensajes. b) Explique por qué un protocolo que entrega datagramas sin conexión **no puede** implementar control de congestión, y qué construcción tecnológica reciente resuelve esa carencia.

T5 (8 puntos). Un desarrollador afirma: "*mandé un mensaje con el flag PUSH activado, así que el programa remoto lo recibirá en una llamada al sistema de lectura distinta de las demás*". a) Argumente por qué la afirmación es incorrecta. b) Describa un procedimiento de laboratorio que demuestre empíricamente la falsedad de la afirmación. c) Indique la práctica correcta para garantizar la separación de mensajes en un protocolo de aplicación, y nombre un protocolo de transporte que sí tenga noción nativa de mensaje.

Problemas Prácticos

Problema 1 (30 puntos) — Programación. Un protocolo de transporte de la familia TCP implementa control de congestión y notifica las pérdidas. Cuando detecta un evento adverso, el emisor **reduce su ventana de congestión** y entra en un régimen de crecimiento más conservador hasta volver al anterior.

Primitivas disponibles:

```
void setVentanaCong(int valor);   // fija el tamaño de la ventana de congestión
int  getVentanaCong();            // lee el tamaño actual
void setUmbral(int valor);        // fija el "umbral" que separa los dos regímenes
int  getUmbral();
void setNotificarPerdida();       // marca los próximos segmentos a enviar con la
                                  // bandera que avisa "hubo pérdida"
```

Tipos:

```
Segmento {
  int  puerto_destino, puerto_origen;
  long numero_secuencia;          // número de secuencia del primer byte
  long numero_confirmacion;       // próximo byte esperado por el otro lado
  int  cantidad_datos;            // bytes de datos del segmento
  bool marca_perdida;             // la bandera de "hubo pérdida"
  IP  encapsulated;               // la cabecera IP del paquete que lo trajo
  ...
}
IP {
  IP  origen, destino;
  enum estado_ecn { NO_SOPORTA, SOPORTA, CONGESTION_EXPERIMENTADA }
  ...
}
```

Se pide: - **a) (6 pts)** ¿Qué eventos, en un protocolo **sin** notificaciones explícitas, interpreta como indicio de pérdida? Explique cómo distingue un evento grave de uno leve, y qué valor toma la ventana de congestión en cada uno. - **b) (12 pts)** Implemente `int detectarCongestion(Segmento s)`, que devuelve 1 si hay congestión y 0 si no. **Si se invoca con `s == NULL`, es porque se venció un timer esperando una confirmación.** El protocolo debe: (i) detectar los ACKs duplicados; (ii) resetear el contador cuando hay progreso real; (iii) propagar la marca de "hubo pérdida" si el segmento recibido la trae; (iv) avisar al otro extremo si la cabecera IP recibida viene marcada por un router como congestionada. - **c) (12 pts)** Implemente `void controlarCongestion(int hayCongestion)`. Ante congestión, el protocolo debe dividir por dos la ventana actual y guardar ese valor como **umbral**; fijar la ventana en **un MSS**; y volver al régimen de arranque. Sin congestión: en el régimen de arranque, **sumar un MSS por cada confirmación recibida** (y cambiar de régimen al superar el umbral); en el régimen de evitación, **sumar `MSS² / ventana` por cada confirmación recibida**. - **d) (2 pts)** El protocolo anuncia en cada segmento el tamaño de su ventana de recepción. Explique, en un párrafo, cómo se combina esa ventana de flujo con la de congestión, y qué se rompe si alguien las sumara.

Problema 2 (30 puntos) — Números y diagramas. Dos procesos, **A** (IP `10.0.0.1`, puerto `5000`) y **B** (IP `10.0.0.2`, puerto `80`), mantienen una conexión TCP abierta. El enlace que los une tiene una ventana de transmisión de **32 KB**.

En un instante dado, A envía a B **cinco** segmentos consecutivos de **24 bytes** de datos cada uno, sin datos previos sin confirmar. El **primer** segmento tiene número de secuencia `7000`. Las latencias son aproximadamente uniformes y el valor del timer es **mayor** que el tiempo de llegada de la primera confirmación.

Se pide: - **a) (8 pts)** Escriba el valor del campo número de secuencia de **cada uno** de los cinco segmentos, y el campo número de confirmación de las respuestas de B, asumiendo que **no hay pérdidas** y que B no tiene datos que enviar. Dibuje el intercambio completo. - **b) (10 pts)** Dibuje ahora el intercambio si **se pierde el primer segmento**. Muestre los valores de los campos y explique, con las dos señales de la RFC, **qué dos eventos** pueden provocar la retransmisión, y **qué valor toma la ventana de congestión** en cada uno. - **c) (6 pts)** El campo de longitud de ventana de la cabecera TCP tiene 16 bits. Calcule el **número máximo de bytes** que puede confirmar en cada paquete el segmento de B con el tamaño de ventana indicado, y el **número de segmentos de 24 bytes** que necesitaría para confirmar los 120 bytes del apartado (a). Si el protocolo negotiating *escalado de ventana*, ¿cuánto cambia? - **d) (6 pts)** Un atacante que no conoce el puerto origen de A quiere cerrar esa conexión mandando un paquete RST. Calcule **cuántos números de puerto** debe adivinar, **cuántos números de secuencia** debe probar para

cada uno, y **cuánto tiempo** tardaría con un enlace de **100 Mbps**, sin considerar tiempos de generación de paquetes. Muestre todas las cuentas.

PARTE 4 — RESOLUCIÓN DEL SIMULACRO

> Todo lo que sigue está en `<details>`. **Resolvé primero el enunciado.**

<details>
<summary>▶ T1 — Resolución (10 pts)</summary>
</details>

<details>
<summary>▶ T2 — Resolución (10 pts)</summary>
</details>

<details>
<summary>▶ T3 — Resolución (8 pts)</summary>
</details>

<details>
<summary>▶ T4 — Resolución (10 pts)</summary>
</details>

<details>
<summary>▶ T5 — Resolución (8 pts)</summary>
</details>

<details>
<summary>▶ Problema 1 — Resolución (30 pts)</summary>
</details>

<details>
<summary>▶ Problema 2 — Resolución (30 pts)</summary>
</details>

📌 Tablero de respuestas del simulacro (para el auto control)

Pregunta	Máx.	Lo que no puede faltar
T1	10	los tres mecanismos · las tres causas indistinguibles · Jacobson/Karels · backoff
T2	10	`rwnd` / `cwnd` con quién decide · `min()` justificado · sonda de 1 byte exponencial · 2 pérdidas no congestivas
T3	8	los 11 estados · handshake con `SYN` consumiendo un número · `TIME-WAIT` 2×MSL
T4	10	8 vs 20 bytes · 2-tupla vs 4-tupla · por qué no puede controlar congestión · QUIC reemplaza a TCP
T5	8	PSH no da fronteras · los dos escenarios del laboratorio · `Content-Length` · SCTP
Teoría	**40**	**mínimo 15 para aprobar**
Problema 1	30	los 2 eventos · los 4 requisitos de `detectarCongestion` · los 3 casos de `controlarCongestion` · el `min`
Problema 2	30	los 5 `seq` · los 5 `conf` · los 4 dupACKs · los 2 eventos con `cwnd` · 2^{17} y el tiempo
Total	**100**	**aprobación: 60**

✅ Autoevaluación del bloque

1. ¿Cuántos estados tiene la máquina de TCP y cuáles son los dos que casi todos olvidan?

▶ Respuesta

2. Un segmento recibido tiene `numero_secuencia = 7024` y 24 bytes de datos. ¿Qué valor debe traer la confirmación de B, y qué se confirmó con ella?

▶ Respuesta

3. ¿Por qué la ventana efectiva es `min(cwnd, rwnd)` y no la suma?

▶ Respuesta

4. Un emisor manda un segmento, no recibe ACK y vence el timer. ¿Qué tres cosas pueden estar pasando?

▶ Respuesta

5. Con 3 ACKs duplicados, ¿qué valor toma `cwnd` en Tahoe y en Reno? ¿Y con timeout?

▶ Respuesta

6. ¿Qué es el BDP y por qué importa?

▶ Respuesta

7. Nombra dos razones por las que UDP no puede implementar control de congestión, y qué lo resuelve hoy.

▶ Respuesta

8. ¿Y qué cambió entre 2011 y 2026 en todo el bloque?

▶ Respuesta

🔗 Referencias

- **Examen `sirc1103`** — P1 (10) forwarding vs. routing · P3 (6) traceroute · **P4 (10) ★ 4 segmentos de 20 bytes desde 1100** · P5 (10) retardo end-to-end · Problema 1 (30) ECN + pseudocode · Problema 2 (30) NAT + ARP + subredes
- **Examen `sirc1012`** — P1 (8) DNS y TTL · P2 (8) mux/demux UDP · P3 (8) hub vs. switch · **P4 (8) números de secuencia y timers** · P5 (8) IPv6 y ND · Problema 1 (30) `connectCheaper()` con sockets UDP · Problema 2 (30) MTU, fragmentación y Path MTU
- **Examen `sirc1102`** — P1 (10) confidencialidad/integridad/no repudio · P2 (10) DHCP · P3 (10) campos de TCP y UDP · P4 (10) NAT · Problema 1 (30) VoIP con jitter buffer · Problema 2 (30) IP Móvil

- **Examen** `sirc1002` — P2 (8) forwarding vs. routing · P3 (8) paridad 2D · P4 (8) firma digital · P5 (8) jitter
- **Examen** `sirc0912` — P2 (10) fiabilidad en transporte vs. enlace · P3 (8) "hops" como métrica
- **Examen** `sirc0907` — **P1 (10) flujo vs. congestión + detección + si toda pérdida es congestión** · P2 (8) three-way handshake y cuándo usar UDP · P3 (8) ARP · P4 (8) RPF e IGMP · P5 (6) confidencialidad vs. integridad
- **Examen** `sirc1003` — P1 (30) subredes + ARP · **Problema 2 (30) ataque TCP reset** · P5 (8) TCP en inalámbricas
- **Examen** `sirc0112` — Problema 1: dos conexiones por un router que pierde · Problema 2: el flag PUSH
- **Examen** `sirc0807` — P2 (10) los cuatro algoritmos de control de congestión
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 completo (pág. ~570–620 del PDF)
- `RFC 9293` — TCP (2022, *Internet Standard*) · `RFC 5681` — TCP Congestion Control (2009)
- `RFC 9438` — CUBIC (2023, *Standards Track*) · `RFC 6928` — IW = 10 MSS
- `RFC 2018` — SACK · `RFC 7323` — Timestamps, Window Scale, PAWS · `RFC 3168` + `RFC 9768` — ECN y AccECN · `RFC 9331` — L4S
- `RFC 9000` / `9001` / `9002` — QUIC · `RFC 9114` — HTTP/3 · `RFC 8684` — MPTCPv1
- `PLAN-48-CLASES.md` y `BANCO-EXAMENES.md` — dónde está cada problema y qué se preguntaba
- **Clase siguiente:** 24 — La capa de red: las dos funciones (forwarding vs. routing)